

CS5340 - Uncertainty Modelling in AI

Lecture 6: Parameter Learning with Complete Data

Scribed by: Claude (Anthropic) | Curated By: Assoc. Prof. Lee Gim Hee
Semester 1, AY2026/27 — National University of Singapore

Disclaimer: This scribe was generated by **Claude (Anthropic)** and curated by **Assoc. Prof. Lee Gim Hee**. It is a **supplementary revision aid** that summarizes the lecture slides, which remain the **primary learning material** prepared by the lecturer. Despite curation, the scribe may still contain errors, omissions, or simplifications. Please use it only alongside the official slides, textbook references, and your own lecture notes.

1 Lecture Overview

- **Motivation:** in Lectures 4 and 5 we learned how to do **exact inference** given a DGM/UGM $p(x_1, \dots, x_M \mid \theta)$. This lecture answers the question we kept deferring: **how do we get the unknown parameter θ from fully observed data?**
- **Given:** a set of N independent and identically distributed (i.i.d.) **complete** observations of every random variable, $\{x_{1,1}, \dots, x_{1,N}\}, \dots, \{x_{M,1}, \dots, x_{M,N}\}$.
- **Roadmap:**
 1. Two estimation principles: **maximum likelihood (MLE)** and **maximum a posteriori (MAP)**.
 2. Special cases with a single random variable: the **univariate normal** (continuous) and the **categorical** (discrete) distribution.
 3. **General DGMs:** complete observations, the decomposable log-likelihood, and closed-form estimates for the discrete (tabular) and continuous (linear-Gaussian) cases.
 4. **MRFs:** the gradient has a clamped and an unclamped term, there is **no closed form**, and we need **stochastic maximum likelihood** or **iterative proportional fitting**.
 5. **CRFs:** the partition function depends on the input, making learning $O(N)$ times slower; we use **stochastic gradient descent**.

Learning Outcomes

By the end of this lecture, you should be able to:

1. Compute the unknown parameters of discrete/continuous DGMs using MLE and MAP.
2. Compute the unknown parameters of MRFs using stochastic maximum likelihood, and iterative proportional fitting.
3. Compute the unknown parameters of CRFs using stochastic gradient descent.

2 Two Ways to Learn Parameters

Definition : Maximum Likelihood Estimate (MLE)

As the name suggests, we find the unknown parameters θ that **maximize the likelihood** $p(x_1, \dots, x_M | \theta)$:

$$\hat{\theta} = \arg \max_{\theta} p(x_1, \dots, x_M | \theta) = \arg \max_{\theta} \prod_{i=1}^N p(x_{1,i}, \dots, x_{M,i} | \theta) \quad (\text{i.i.d.})$$

Definition : Maximum a Posteriori (MAP)

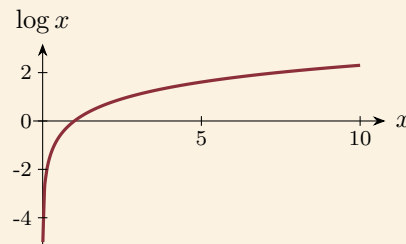
As the name suggests, we find the unknown parameters θ that **maximize the posterior** $p(\theta | x_1, \dots, x_M)$:

$$\begin{aligned} \hat{\theta} &= \arg \max_{\theta} p(\theta | x_1, \dots, x_M) = \arg \max_{\theta} \frac{p(x_1, \dots, x_M | \theta) p(\theta)}{p(x_1, \dots, x_M)} && (\text{Bayes' rule}) \\ &= \arg \max_{\theta} \frac{\prod_{i=1}^N p(x_{1,i}, \dots, x_{M,i} | \theta) p(\theta)}{p(x_1, \dots, x_M)} && (\text{i.i.d.}) \\ &= \arg \max_{\theta} \prod_{i=1}^N p(x_{1,i}, \dots, x_{M,i} | \theta) p(\theta), \end{aligned}$$

where $p(x_1, \dots, x_M)$ is removed because it is **independent of θ** .

Key Rule: Why We Maximize the Logarithm

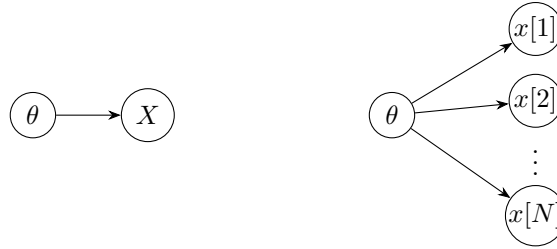
- The logarithm is a **monotonic** transformation, so the position of the peak **stays in the same place**.
- But the log-likelihood turns products into sums, and so is **much easier to work with** (and avoids numerical underflow).



(Image source: S. Prince, “Computer Vision: Models, Learning, and Inference”.)

2.1 Special Case: A Single-Variable DGM

We first learn the unknown parameter θ of a **single random variable** DGM $p(x | \theta)$. For N i.i.d. observations $X : \{x[1], \dots, x[N]\}$, the DGM becomes an augmentation of N **disconnected replicates** of X , all sharing the same parameter:



3 Special Case (Continuous DGM): Univariate Normal Distribution

Problem: fit a univariate normal distribution to scalar data $X : \{x[1], \dots, x[N]\}$, where

$$p(x \mid \theta) = \text{Norm}_x[\mu, \sigma^2] = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right),$$

i.e. find the two unknown parameters $\theta = (\mu, \sigma^2)$.

3.1 Approach 1: Maximum Likelihood Estimation

Worked Example: MLE for the Univariate Normal

The likelihood is given by the pdf, $p(x \mid \mu, \sigma^2) = \prod_{i=1}^N \text{Norm}_{x[i]}[\mu, \sigma^2]$. Maximizing its logarithm,

$$\begin{aligned} \hat{\mu}, \hat{\sigma}^2 &= \arg \max_{\mu, \sigma^2} \sum_{i=1}^N \log [\text{Norm}_{x[i]}[\mu, \sigma^2]] \\ &= \arg \max_{\mu, \sigma^2} \underbrace{\left[-0.5N \log[2\pi] - 0.5N \log \sigma^2 - 0.5 \sum_{i=1}^N \frac{(x[i] - \mu)^2}{\sigma^2} \right]}_L. \end{aligned}$$

Maximization can be done in **closed form** by taking derivatives and equating them to zero:

$$\begin{aligned} \frac{\partial L}{\partial \mu} &= \sum_{i=1}^N \frac{(x[i] - \mu)}{\sigma^2} = \frac{\sum_{i=1}^N x[i]}{\sigma^2} - \frac{N\mu}{\sigma^2} = 0, & \frac{\partial L}{\partial \sigma^2} &= -\frac{N}{\sigma^2} + \sum_{i=1}^N \frac{(x[i] - \mu)^2}{\sigma^4} = 0 \\ \Rightarrow \hat{\mu} &= \frac{\sum_{i=1}^N x[i]}{N} = \bar{x}, & \Rightarrow \hat{\sigma}^2 &= \frac{\sum_{i=1}^N (x[i] - \hat{\mu})^2}{N}. \end{aligned}$$

The MLE of the mean is the **sample mean**, and the MLE of the variance is the **sample variance** about it.

Key Rule: Maximum Likelihood for a Normal = Least Squares

Keeping only the terms that involve μ ,

$$\begin{aligned}\hat{\mu} &= \arg \max_{\mu} \left[-0.5N \log[2\pi] - 0.5N \log \sigma^2 - 0.5 \sum_{i=1}^N \frac{(x[i] - \mu)^2}{\sigma^2} \right] \\ &= \arg \max_{\mu} \left[- \sum_{i=1}^N (x[i] - \mu)^2 \right] = \arg \min_{\mu} \sum_{i=1}^N (x[i] - \mu)^2,\end{aligned}$$

which is exactly the **least squares** fitting criterion. Fitting by least squares is therefore the same as assuming normally distributed (equal-variance) noise and maximizing the likelihood.

3.2 Approach 2: Maximum a Posteriori**Definition : Conjugate Prior – Normal-Inverse-Gamma**

MAP maximizes $\prod_{i=1}^N p(x[i] | \theta) p(\theta)$, i.e. **likelihood** $\times$ **prior**. For the normal likelihood with unknown (μ, σ^2) , the **conjugate prior** is the normal-inverse-gamma distribution with hyperparameters $(\alpha, \beta, \gamma, \delta)$:

$$p(\mu, \sigma^2) = \text{NormInvGam}_{\mu, \sigma^2}[\alpha, \beta, \gamma, \delta] = \frac{\sqrt{\gamma}}{\sigma \sqrt{2\pi}} \frac{\beta^\alpha}{\Gamma[\alpha]} \left(\frac{1}{\sigma^2} \right)^{\alpha+1} \exp \left[-\frac{2\beta + \gamma(\delta - \mu)^2}{2\sigma^2} \right].$$

A prior is **conjugate** to a likelihood when the posterior has the same functional form as the prior, which is what keeps the MAP estimate available in closed form.

Worked Example: MAP for the Univariate Normal

$$\begin{aligned}\hat{\mu}, \hat{\sigma}^2 &= \arg \max_{\mu, \sigma^2} \prod_{i=1}^N \text{Norm}_{x[i]}[\mu, \sigma^2] \text{NormInvGam}_{\mu, \sigma^2}[\alpha, \beta, \gamma, \delta] \\ &= \arg \max_{\mu, \sigma^2} \underbrace{\left[\sum_{i=1}^N \log [\text{Norm}_{x[i]}[\mu, \sigma^2]] + \log [\text{NormInvGam}_{\mu, \sigma^2}[\alpha, \beta, \gamma, \delta]] \right]}_L.\end{aligned}$$

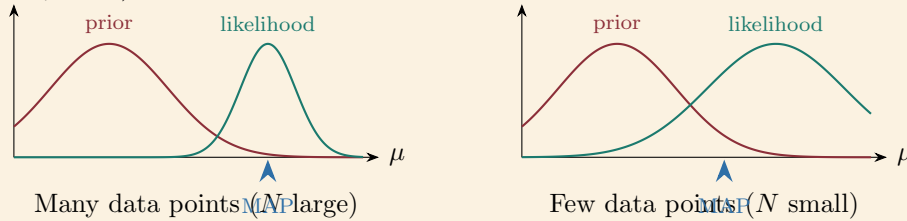
Taking derivatives, $\partial L / \partial \mu = 0$ and $\partial L / \partial \sigma^2 = 0$, we get

$$\hat{\mu} = \frac{\sum_i x[i] + \gamma \delta}{N + \gamma} = \frac{N \bar{x} + \gamma \delta}{N + \gamma}, \quad \hat{\sigma}^2 = \frac{\sum_i (x[i] - \hat{\mu})^2 + 2\beta + \gamma(\delta - \hat{\mu})^2}{N + 3 + 2\alpha}.$$

The MAP mean is a **weighted average** of the sample mean $\bar{x}$ (weight N) and the prior mean δ (weight γ): the hyperparameter γ acts like a number of “virtual” observations placed at δ .

Key Rule: Comparing MLE and MAP

- **More data points** → MAP moves closer to the **MLE** (the likelihood dominates; in $\hat{\mu}$, $N \gg \gamma$).
- **Fewer data points** → MAP moves closer to the **MP** (the maximum of the prior; when $N \rightarrow 0$, $\hat{\mu} \rightarrow \delta$).



(Schematic drawn by the scribes; the slides show the corresponding posterior heat maps over (μ, σ^2) for 50, 5 and 1 data points, from S. Prince.) With 50 points the MAP estimate sits almost on top of the MLE; with a single point it is pulled most of the way back to the prior.

4 Special Case (Discrete DGM): Univariate Categorical Distribution

Problem: fit a categorical distribution with K categories to data $\mathbf{X} : \{\mathbf{x}[1], \dots, \mathbf{x}[N]\}$, where each $\mathbf{x}[i] = \mathbf{e}_k$ is a **one-hot** vector, e.g. $[0, 0, 0, 1, 0, 0]$ with $K = 6$. Recall

$$p(\mathbf{X} = \mathbf{e}_k \mid \theta) = \text{Cat}_{\mathbf{x}}[\lambda] = \prod_{k=1}^K \lambda_k^{x_k} = \lambda_k,$$

so the goal is to find $\theta = \{\lambda_1, \dots, \lambda_K\}$ with $\lambda_k \in [0, 1]$ and $\sum_k \lambda_k = 1$.

Worked Example: MLE for the Categorical Distribution

The likelihood of the data set is

$$p(\mathbf{x} \mid \lambda) = \prod_{i=1}^N \text{Cat}_{\mathbf{x}[i]}[\lambda_{1...K}] = \prod_{i=1}^N \prod_{k=1}^K \lambda_k^{x_{ik}} = \prod_{k=1}^K \lambda_k^{N_k},$$

where $x_{ik} \in \{0, 1\}$ is the k th element of $\mathbf{x}_i$ and N_k is the **number of times we observed bin k** . Hence

$$\hat{\lambda}_{1...K} = \arg \max_{\lambda_{1...K}} \prod_{k=1}^K \lambda_k^{N_k}, \quad \text{s.t.} \quad \sum_k \lambda_k = 1.$$

Applying the log and a **Lagrange multiplier** ν for the constraint gives the auxiliary function

$$\mathcal{L} = \sum_{k=1}^K N_k \log \lambda_k + \nu \left(\sum_{k=1}^K \lambda_k - 1 \right).$$

Taking $\partial \mathcal{L} / \partial \lambda_k = 0$ and $\partial \mathcal{L} / \partial \nu = 0$ and solving,

$$\hat{\lambda}_k = \frac{N_k}{\sum_{m=1}^K N_m}$$

i.e. the **normalized counts** of the number of times we observed bin k .

Worked Example: MAP for the Categorical Distribution

The **conjugate prior** of the categorical likelihood is the **Dirichlet** distribution,

$$p(\lambda_1, \dots, \lambda_K) = \text{Dir}_{\lambda_{1\dots K}}[\alpha_1, \dots, \alpha_K] = \frac{\Gamma\left[\sum_{k=1}^K \alpha_k\right]}{\prod_{k=1}^K \Gamma[\alpha_k]} \prod_{k=1}^K \lambda_k^{\alpha_k-1}, \quad \lambda_k \in [0, 1], \sum_k \lambda_k = 1.$$

Since the normalizing factor of the Dirichlet is **independent of** λ , it can be ignored:

$$\hat{\lambda}_{1\dots K} = \arg \max_{\lambda_{1\dots K}} \prod_{k=1}^K \lambda_k^{N_k} \prod_{k=1}^K \lambda_k^{\alpha_k-1} = \arg \max_{\lambda_{1\dots K}} \prod_{k=1}^K \lambda_k^{N_k+\alpha_k-1}, \quad \text{s.t. } \sum_k \lambda_k = 1.$$

With the log and a Lagrange multiplier, $\mathcal{L} = \sum_{k=1}^K (N_k + \alpha_k - 1) \log \lambda_k + \nu (\sum_k \lambda_k - 1)$, giving

$$\hat{\lambda}_k = \frac{N_k + \alpha_k - 1}{\sum_{m=1}^K (N_m + \alpha_m - 1)}$$

This is the **same result as MLE** when the prior is uniform, $\alpha_{1\dots K} = 1$. The hyperparameters act as **pseudo-counts**: $\alpha_k - 1$ observations of bin k added before seeing any data.

Worked Example: Pseudo-Counts in Numbers (Scribe's Example)

Not from the slides; added to show why the prior matters. Suppose $K = 3$ and we observe the counts $(N_1, N_2, N_3) = (3, 1, 0)$. Then

$$\hat{\lambda}^{\text{MLE}} = \left(\frac{3}{4}, \frac{1}{4}, 0\right), \quad \hat{\lambda}^{\text{MAP}} \text{ with } \alpha = (2, 2, 2) : \left(\frac{3+1}{7}, \frac{1+1}{7}, \frac{0+1}{7}\right) = \left(\frac{4}{7}, \frac{2}{7}, \frac{1}{7}\right).$$

MLE assigns probability **zero** to category 3 simply because it was never observed – any future data point in that bin would make the whole data set impossible. The Dirichlet prior **smooths** the estimate away from zero.

5 Parameter Learning: DGMs

Definition : Setup and Complete Observations

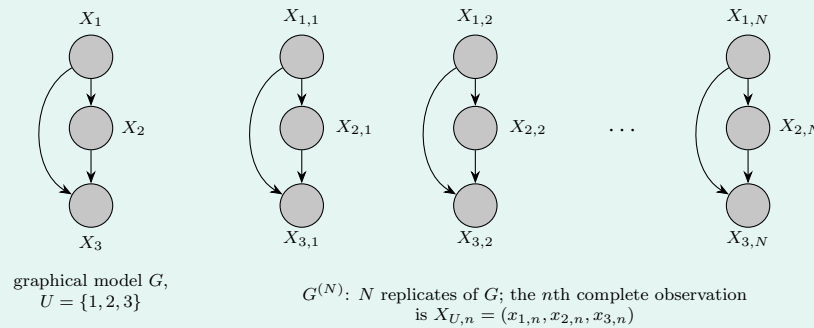
Let $G = (U, E)$ be a directed graph, X_u the random variable at node $u \in U$, and x_u a realization. To each node we associate a **local conditional probability distribution** $p(x_u \mid x_{\pi_u}, \theta_u)$, where π_u are the indices of the parents of u and θ_u is a parameter vector. The overall probability is the product of the local probabilities,

$$p(x_U \mid \theta) = \prod_{u \in U} p(x_u \mid x_{\pi_u}, \theta_u), \quad \theta = \{\theta_1, \dots, \theta_{|U|}\}.$$

A **complete observation** is an assignment of values to **ALL** of the random variables X_U in the model (shaded nodes below).

Worked Example: Replicating the Graph for N Observations

For N i.i.d. observations the graphical model becomes an augmentation of N **disconnected replicates** of G , written $G^{(N)} = (U^{(N)}, E^{(N)})$. Nodes are indexed by a **pair of labels** (u, n) : $u \in U$ designates a node of G and $n \in \{1, \dots, N\}$ the replication number.



(Image source: M. I. Jordan, 2002.)

Key Rule: The Log-Likelihood Decomposes over Nodes

Writing the entire observed data set as $\mathcal{D} = (x_{U,1}, x_{U,2}, \dots, x_{U,N})$, the probability model for $G^{(N)}$ is

$$\begin{aligned} p(\mathcal{D} \mid \theta) &= \prod_n p(x_{U,n} \mid \theta) = \prod_n \prod_u p(x_{u,n} \mid x_{\pi_u,n}, \theta_u) \quad (\text{i.i.d.}) \\ \implies \log p(\mathcal{D} \mid \theta) &= \sum_n \sum_u \log p(x_{u,n} \mid x_{\pi_u,n}, \theta_u). \end{aligned}$$

Maximizing over one node's parameter θ_u ,

$$\arg \max_{\theta_u} \log p(\mathcal{D} \mid \theta) = \arg \max_{\theta_u} \sum_n \log p(x_{u,n} \mid x_{\pi_u,n}, \theta_u),$$

because **all terms that do not involve θ_u can be ignored**. This implies that it is sufficient to estimate θ_u from the **local subset of observations**, i.e. the **sufficient statistics**

$$\{x_{u,n}, x_{\pi_u,n}\}_{n=1}^N.$$

Learning a DGM with complete data therefore decomposes into $|U|$ independent local learning problems.

Key Rule: MAP for DGMs

Taking the maximum a posteriori over θ_u gives

$$\arg \max_{\theta_u} \log p(\theta \mid \mathcal{D}) = \arg \max_{\theta_u} \left\{ \sum_n \log p(x_{u,n} \mid x_{\pi_u,n}, \theta_u) + \log p(\theta_u) \right\},$$

where $p(\theta_u)$ is the **conjugate prior** of the likelihood distribution. The same decomposition holds, one local problem per node.

5.1 Discrete Case: Tabular CPDs

Definition : Categorical Likelihood for a Node

$$p(x_u \mid x_{\pi_u}, \lambda_u) = \text{Cat}_{x_u \mid x_{\pi_u}}[\lambda_u] = \prod_{c=1}^C \prod_{k=1}^K \lambda_{uck}^{x_{uck}} = \lambda_{uck}, \quad \text{s.t.} \quad \sum_k \lambda_{uck} = 1,$$

with parameter $\lambda_u = \{\lambda_{u11}, \dots, \lambda_{uck}, \dots, \lambda_{uCK}\}$, where C is the total number of states that X_{π_u} takes and K the total number of states that X_u takes; $x_{uck} = 1$ when $X_u = k$ and $X_{\pi_u} = c$, and 0 otherwise.

Worked Example: MLE for a Tabular CPD

Putting the likelihood into the maximum log-likelihood,

$$\arg \max_{\lambda_{uc1}, \dots, \lambda_{ucK}} \sum_{n=1}^N \sum_{c=1}^C \sum_{k=1}^K \log \lambda_{uck}^{x_{uck,n}}, \quad \text{s.t.} \quad \sum_k \lambda_{uck} = 1.$$

The **sum over c is dropped** because we optimize the parameters of each configuration of X_{π_u} separately:

$$\Rightarrow \arg \max_{\lambda_{uc1}, \dots, \lambda_{ucK}} \sum_{n=1}^N \sum_{k=1}^K \log \lambda_{uck}^{x_{uck,n}} \Rightarrow \arg \max_{\lambda_{uc1}, \dots, \lambda_{ucK}} \sum_{k=1}^K \log \lambda_{uck}^{N_{uck}},$$

where N_{uck} is the **number of times we observe** ($x_u = k, x_{\pi_u} = c$). With a Lagrange multiplier,

$$\mathcal{L} = \sum_{k=1}^K N_{uck} \log \lambda_{uck} + \nu \left(\sum_k \lambda_{uck} - 1 \right) \Rightarrow \boxed{\hat{\lambda}_{uck} = \frac{N_{uck}}{\sum_{m=1}^K N_{ucm}}}$$

the **normalized counts** of the number of times we observed $x_u = k, x_{\pi_u} = c$. (The slide writes the denominator as $\sum_m N_m$; the counts are those of the same node and parent configuration, N_{ucm} .)

Worked Example: MAP for a Tabular CPD

The conjugate prior is again the **Dirichlet**, with K hyperparameters $\alpha_{uck} > 1$ for each variable X_u and each state $X_{\pi_u} = c$:

$$p(\lambda_{uc1}, \dots, \lambda_{ucK}) = \text{Dir}_{\lambda_{uc1}, \dots, \lambda_{ucK}}[\alpha_{uc1}, \dots, \alpha_{ucK}] = \frac{\Gamma\left[\sum_{k=1}^K \alpha_{uck}\right]}{\prod_{k=1}^K \Gamma[\alpha_{uck}]} \prod_{k=1}^K \lambda_{uck}^{\alpha_{uck}-1}.$$

Substituting into $\arg \max_{\lambda_u} \{\sum_n \log p(x_{u,n} | x_{\pi_u,n}, \lambda_u) + \log p(\lambda_u)\}$ and dropping the term independent of λ_{uck} ,

$$= \arg \max_{\lambda_{uc1}, \dots, \lambda_{ucK}} \sum_k \log \lambda_{uck}^{N_{uck} + \alpha_{uck} - 1}, \quad \text{s.t. } \sum_k \lambda_{uck} = 1.$$

With a Lagrange multiplier, $\mathcal{L} = \sum_{k=1}^K (N_{uck} + \alpha_{uck} - 1) \log \lambda_{uck} + \nu (\sum_k \lambda_{uck} - 1)$, so

$$\hat{\lambda}_{uck} = \frac{N_{uck} + \alpha_{uck} - 1}{\sum_{m=1}^K (N_{ucm} + \alpha_{ucm} - 1)}$$

(The slide's auxiliary function prints λ_{uck} where the pseudo-count α_{uck} is meant.)

5.2 Continuous Case: Linear-Gaussian CPDs

Definition : Linear-Gaussian Likelihood for a Node

$$\begin{aligned} p(x_u | x_{\pi_u}, \theta_{x_u|x_{\pi_u}}) &= \text{Norm}_{x_u|x_{\pi_u}}[w_{u0}, \dots, w_{uC}, \sigma_u^2] \\ &= \frac{1}{\sqrt{2\pi\sigma_u^2}} \exp\left(-0.5 \frac{\left(x_u - \left(\sum_{c \in x_{\pi_u}} w_{uc}x_{uc} + w_{u0}\right)\right)^2}{\sigma_u^2}\right), \end{aligned}$$

with parameters $\theta_{x_u|x_{\pi_u}} = \{w_{u0}, \dots, w_{uC}, \sigma_u^2\}$. The mean $\mu_u = \sum_{c \in x_{\pi_u}} w_{uc}x_{uc} + w_{u0}$ is a **weighted sum of the parent nodes**, and C is the total number of parents.

Worked Example: MLE for a Linear-Gaussian CPD

$$\begin{aligned} &\arg \max_{\theta_{x_u|x_{\pi_u}}} \sum_{n=1}^N \log p(x_{u,n} | x_{\pi_u,n}, \theta_{x_u|x_{\pi_u}}) \\ &= \arg \max_{\theta_{x_u|x_{\pi_u}}} \underbrace{\sum_{n=1}^N \left[-\frac{1}{2} \log(2\pi\sigma_u^2) - \frac{1}{2\sigma_u^2} \left(x_{u,n} - \left(\sum_{c \in x_{\pi_u}} w_{uc}x_{uc,n} + w_{u0} \right) \right)^2 \right]}_L. \end{aligned}$$

Taking the derivative of L w.r.t. w_{u0}, w_{uc} and equating to zero gives $C + 1$ equations:

$$\begin{aligned}\frac{\partial L}{\partial w_{u0}} &= \sum_{n=1}^N \left(x_{u,n} - (w_{u1}x_{u1,n} + \dots + w_{uC}x_{uC,n} + w_{u0}) \right) = 0 \\ \frac{\partial L}{\partial w_{u1}} &= \sum_{n=1}^N \left(x_{u,n} - (w_{u1}x_{u1,n} + \dots + w_{uC}x_{uC,n} + w_{u0}) \right) x_{u1,n} = 0 \\ &\vdots \\ \frac{\partial L}{\partial w_{uC}} &= \sum_{n=1}^N \left(x_{u,n} - (w_{u1}x_{u1,n} + \dots + w_{uC}x_{uC,n} + w_{u0}) \right) x_{uC,n} = 0\end{aligned}$$

which are solved for the $C + 1$ unknowns $w_{u0}, w_{u1}, \dots, w_{uC}$ (these are the **normal equations** of linear least squares). Finally, take the derivative of L w.r.t. σ_u^2 and equate to zero to solve for σ_u^2 .

Definition : Priors for the Linear-Gaussian Parameters

We factor the prior of $\theta_{x_u|x_{\pi_u}} = \{w_{u0}, \dots, w_{uC}, \sigma_u^2\}$ as

$$p(w_{u0}, \dots, w_{uC}, \sigma_u^2) = p(\sigma_u^2) p(w_{u0}, \dots, w_{uC} \mid \sigma_u^2) = p(\sigma_u^2) \prod_{c \in x_{\pi_u}} p(w_{uc} \mid \sigma_u^2) \quad (\text{Naïve Bayes}),$$

where each weight follows a **univariate normal** with hyperparameter μ_u ,

$$p(w_{uc} \mid \mu_u, \sigma_u^2) = \frac{1}{\sqrt{2\pi\sigma_u^2}} \exp\left(-\frac{(w_{uc} - \mu_u)^2}{2\sigma_u^2}\right) = \text{Norm}_{w_{uc}}[\mu_u, \sigma_u^2],$$

and the variance follows an **inverse gamma** distribution,

$$p(\sigma_u^2 \mid \alpha_u, \beta_u) = \frac{\beta_u^{\alpha_u}}{\Gamma(\alpha_u)} (\sigma_u^2)^{-\alpha_u-1} \exp\left(-\frac{\beta_u}{\sigma_u^2}\right) = \text{InvGam}_{\sigma_u^2}[\alpha_u, \beta_u],$$

where (α_u, β_u) are hyperparameters describing the **shape** and **scale** of the distribution, $\alpha_u > 0$, $\beta_u > 0$, and $\Gamma(\alpha_u)$ is the gamma function.

Worked Example: MAP for a Linear-Gaussian CPD

Putting likelihood and conjugate priors into $\arg \max \{\sum_n \log p(x_{u,n} \mid x_{\pi_u,n}, \theta) + \log p(\theta)\}$:

$$\arg \max_{\theta_{x_u|x_{\pi_u}}} \{L\}, \quad \text{where}$$

$$L = \sum_n \log \text{Norm}_{x_{u,n}|x_{\pi_u,n}}[w_{u0}, \dots, w_{uC}, \sigma_u^2] + \log \text{InvGam}_{\sigma_u^2}[\alpha_u, \beta_u] + \sum_c \log \text{Norm}_{w_{uc}}[\mu_u, \sigma_u^2].$$

As in the MLE case, $\partial L / \partial w_{u0} = 0$, $\partial L / \partial w_{u1} = 0$, $\dots$, $\partial L / \partial w_{uC} = 0$ give $C + 1$ equations for the $C + 1$ unknowns $\{w_{u0}, \dots, w_{uC}\}$, and finally $\partial L / \partial \sigma_u^2 = 0$ is solved for σ_u^2 . The normal prior on the weights adds a quadratic penalty $\propto (w_{uc} - \mu_u)^2$, i.e. **ridge-like regularization** of the regression weights toward μ_u .

Summary: Closed-Form Estimates for DGMs

Model	MLE	MAP (conjugate prior)
Univariate normal	$\hat{\mu} = \bar{x},$ $\hat{\sigma}^2 = \frac{1}{N} \sum_i (x[i] - \hat{\mu})^2$	$\hat{\mu} = \frac{N\bar{x} + \gamma\delta}{N + \gamma}$ (normal-inverse-gamma)
Categorical	$\hat{\lambda}_k = N_k / \sum_m N_m$	$\hat{\lambda}_k = \frac{N_k + \alpha_k - 1}{\sum_m (N_m + \alpha_m - 1)}$ (Dirichlet)
DGM, tabular CPD	$\hat{\lambda}_{uck} = N_{uck} / \sum_m N_{ucm}$	$\hat{\lambda}_{uck} = \frac{N_{uck} + \alpha_{uck} - 1}{\sum_m (N_{ucm} + \alpha_{ucm} - 1)}$ (Dirichlet)
DGM, linear-Gaussian	$C + 1$ normal equations for w , then σ_u^2	same equations with normal prior on w , inverse-gamma on σ_u^2

Take-away: with **complete data** a DGM's log-likelihood decomposes node by node, so every estimate above is **local** and **closed-form** – counting for discrete nodes, least squares for linear-Gaussian nodes.

6 Parameter Learning: UGM (MRF)

Definition : MRF in Log-Linear Form

Consider an MRF where c indexes the cliques:

$$p(\mathbf{y} \mid \boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} \exp \left(\sum_c \boldsymbol{\theta}_c^T \phi_c(\mathbf{y}) \right).$$

The **scaled log-likelihood** of N observations is

$$\ell(\boldsymbol{\theta}) \triangleq \frac{1}{N} \sum_i \log p(\mathbf{y}_i \mid \boldsymbol{\theta}) = \frac{1}{N} \sum_i \left[\sum_c \boldsymbol{\theta}_c^T \phi_c(\mathbf{y}_i) - \log Z(\boldsymbol{\theta}) \right].$$

Key Rule: Convexity and the Gradient

Since MRFs are in the **exponential family**, this function is **convex** in θ , so it has a **unique global maximum** that we can find with **gradient-based optimizers**. The derivative for the weights of a particular clique c is

$$\frac{\partial \ell}{\partial \boldsymbol{\theta}_c} = \frac{1}{N} \sum_i \left[\phi_c(\mathbf{y}_i) - \frac{\partial}{\partial \boldsymbol{\theta}_c} \log Z(\boldsymbol{\theta}) \right].$$

Proof: The Derivative of the Log Partition Function

Claim: $\frac{\partial \log Z(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_c} = \mathbb{E}[\phi_c(\mathbf{y}) \mid \boldsymbol{\theta}] = \sum_{\mathbf{y}} \phi_c(\mathbf{y}) p(\mathbf{y} \mid \boldsymbol{\theta})$, i.e. it is the **expectation of the c th feature under the model**.

With $Z(\boldsymbol{\theta}) = \sum_{\mathbf{y}} \exp(\sum_c \boldsymbol{\theta}_c^T \phi_c(\mathbf{y}))$,

$$\begin{aligned} \frac{\partial \log Z(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_c} &= \frac{1}{Z(\boldsymbol{\theta})} \frac{\partial Z(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_c}, \quad \text{where} \quad \frac{\partial Z(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_c} = \sum_{\mathbf{y}} \exp(\sum_c \boldsymbol{\theta}_c^T \phi_c(\mathbf{y})) \phi_c(\mathbf{y}) \\ \Rightarrow \frac{\partial \log Z(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_c} &= \frac{1}{Z(\boldsymbol{\theta})} \sum_{\mathbf{y}} \phi_c(\mathbf{y}) \exp(\sum_c \boldsymbol{\theta}_c^T \phi_c(\mathbf{y})) \\ &= \sum_{\mathbf{y}} \phi_c(\mathbf{y}) \underbrace{\frac{1}{Z(\boldsymbol{\theta})} \exp(\sum_c \boldsymbol{\theta}_c^T \phi_c(\mathbf{y}))}_{p(\mathbf{y} \mid \boldsymbol{\theta})} = \sum_{\mathbf{y}} \phi_c(\mathbf{y}) p(\mathbf{y} \mid \boldsymbol{\theta}). \end{aligned}$$

■

Key Rule: Clamped minus Unclamped (Moment Matching)

Hence the gradient of the log-likelihood is

$$\frac{\partial \ell}{\partial \theta_c} = \underbrace{\left[\frac{1}{N} \sum_i \phi_c(\mathbf{y}_i) \right]}_{\text{clamped term}} - \underbrace{\mathbb{E}[\phi_c(\mathbf{y})]}_{\text{unclamped / contrastive term}} = \mathbb{E}_{p_{\text{emp}}}[\phi_c(y)] - \mathbb{E}_{p(y|\theta)}[\phi_c(y)].$$

- **Clamped term:** y is **fixed to its observed values**; it is the expected feature vector under the **empirical distribution**, $\mathbb{E}_{p_{\text{emp}}}[\phi_c(y)] = \frac{1}{N} \sum_{i=1}^N \phi_c(y_i)$.
- **Unclamped/contrastive term:** y is a **free variable**; it is the expected feature vector under the **model's distribution**.
- The unclamped term **requires inference** in the model, **once per gradient step**, which makes UGM learning **much slower** than DGM learning.

At the optimum the gradient is zero, so the two expectations **match**: $\mathbb{E}_{p_{\text{emp}}}[\phi_c(y)] = \mathbb{E}_{p(y|\theta)}[\phi_c(y)]$.

Key Difficulty: No Closed-Form Solution

$\mathbb{E}_{p(y|\theta)}[\phi_c(y)] = \sum_y \phi_c(y) p(y | \theta)$ **cannot be evaluated in closed form** in terms of the unknown parameters θ , because $p(y | \theta)$ itself depends on θ through $Z(\theta)$.

We CANNOT solve for the parameters θ in closed form!

Solution: use **gradient-based optimizers**. But the gradient requires a **sum over all states of y** , which can be intractable, so learning is intractable too – unless we combine **approximate inference** with gradient-based learning. Two such methods follow.

6.1 Method 1: Stochastic Maximum Likelihood**Definition : Stochastic Maximum Likelihood (SML)**

A **stochastic gradient descent** method. We iteratively update the parameter at step k using the parameter and gradient from the previous step,

$$\theta_{k+1} \leftarrow \theta_k - \eta g_k,$$

where η is the **step size** (learning rate) and $g_k \approx \partial \ell / \partial \theta_c$ is the gradient, **approximated with Markov Chain Monte Carlo (MCMC) sampling** (covered in Lecture 10).

Algorithm: Stochastic Maximum Likelihood for Fitting an MRF (Murphy)

- 1: Initialize weights θ randomly
- 2: $k = 0$, $\eta = 1$
- 3: **for** each epoch **do**
- 4: **for** each minibatch of size B **do** $\triangleright$ split the data y_i , $i = 1 \dots N$ into sets of size B
- 5: **for** each sample $s = 1 : S$ **do**
- 6: Sample $\mathbf{y}^{s,k} \sim p(\mathbf{y} | \theta_k)$ $\triangleright$ S samples from the model, parameter fixed at θ_k
- 7: **end for**

```

8:       $\hat{E}(\phi(\mathbf{y})) = \frac{1}{S} \sum_{s=1}^S \phi(\mathbf{y}^{s,k})$  ▷ approximate the unclamped term
9:      for each training case  $i$  in minibatch do
10:          $\mathbf{g}_{ik} = \phi(\mathbf{y}_i) - \hat{E}(\phi(\mathbf{y}))$  ▷ approximated gradient,  $g_k \approx \partial \ell / \partial \theta$ 
11:      end for
12:       $\mathbf{g}_k = \frac{1}{B} \sum_{i \in B} \mathbf{g}_{ik}$ 
13:       $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \mathbf{g}_k$  ▷ stochastic gradient descent update step
14:       $k = k + 1$ 
15:      Decrease step size  $\eta$ 
16:  end for
17: end for

```

Scribe's note on signs: as written, $\mathbf{g}_k$ is the gradient of the **log-likelihood** (clamped – unclamped), which we want to **maximize**; the minus sign in the update is the convention for **descending the negative log-likelihood**. Keep the two conventions consistent in an implementation: ascend ℓ , or descend $-\ell$.

6.2 Method 2: Iterative Proportional Fitting

Definition : Gradient in Terms of the Potentials

An alternative derivation of the gradient in terms of $\psi_c(y_c)$ (tabular potentials rather than log-linear weights):

$$\ell = \log \prod_i \frac{1}{Z} \prod_c \psi_c(y_{i,c}) = \sum_i \sum_c \log \psi_c(y_{i,c}) - N \log Z = \sum_c \sum_{y_c} N(y_c) \log \psi_c(y_c) - N \log Z,$$

where $N(y_c) = \sum_i \delta(y_c, y_{i,c})$ is the **number of times clique c is in configuration y_c** in the data. Therefore

$$\frac{\partial \ell}{\partial \psi_c(y_c)} = \frac{N(y_c)}{\psi_c(y_c)} - N \underbrace{\frac{\partial}{\partial \psi_c(y_c)} \log Z}_{\text{no closed form}}.$$

Proof: Derivative of the Log Partition Function w.r.t. a Potential

$$\begin{aligned} \frac{\partial}{\partial \psi_c(y_c)} \log Z &= \frac{1}{Z} \frac{\partial Z}{\partial \psi_c(y_c)} = \frac{1}{Z} \frac{\partial}{\partial \psi_c(y_c)} \sum_y \prod_D \psi_D(y_D) \\ &= \frac{1}{Z} \sum_y \delta(y, y_c) \frac{\partial}{\partial \psi_c(y_c)} \prod_D \psi_D(y_D) = \frac{1}{Z} \sum_y \delta(y, y_c) \prod_{D \setminus c} \psi_D(y_D) \\ &= \sum_y \delta(y, y_c) \frac{1}{\psi_c(y_c)} \underbrace{\frac{1}{Z} \prod_D \psi_D(y_D)}_{p(y)} = \frac{1}{\psi_c(y_c)} \underbrace{\sum_y \delta(y, y_c) p(y)}_{\sum_{y \setminus y_c} p(y)} = \frac{p(y_c | \psi)}{\psi_c(y_c)}. \end{aligned}$$

The result is **conditioned on all potentials ψ** , since $p(y_c | \psi)$ is obtained by **marginalization of the full distribution $p(y)$** – i.e. it requires inference. ■

Key Rule: The IPF Fixed-Point Equation

Putting the derivative back into the gradient and equating to zero,

$$\frac{N(y_c)}{\psi_c(y_c)} - N \frac{p(y_c | \psi)}{\psi_c(y_c)} = 0 \quad \implies \quad \underbrace{\frac{N(y_c)}{N}}_{p_{\text{emp}}(y_c)} \frac{1}{\psi_c(y_c)} = \frac{p(y_c | \psi)}{\psi_c(y_c)}.$$

We can solve for $\psi_c(y_c)$ iteratively using the **fixed-point equation**

$$\psi_c^{t+1}(\mathbf{y}_c) = \psi_c^t(\mathbf{y}_c) \times \frac{p_{\text{emp}}(\mathbf{y}_c)}{p(\mathbf{y}_c | \psi^t)}$$

where the division and multiplication are **element-wise** over the clique's configurations. Each update scales a clique potential up where the data are more frequent than the current model predicts, and down where they are less frequent; at a fixed point the model's clique marginals **match** the empirical ones.

Algorithm: Iterative Proportional Fitting for Tabular MRFs (Murphy)

- 1: Initialize $\psi_c = 1$ for $c = 1 : C$
- 2: **repeat**
- 3: **for** $c = 1 : C$ **do**
- 4: $p_c = p(\mathbf{y}_c | \psi)$ ▷ marginalize with the current ψ : $\sum_{y \setminus y_c} p(y)$
- 5: $\hat{p}_c = p_{\text{emp}}(\mathbf{y}_c)$ ▷ empirical probability of the current clique c
- 6: $\psi_c = \psi_c * \frac{\hat{p}_c}{p_c}$ ▷ iterative proportional fitting (element-wise)
- 7: **end for**
- 8: **until** converged

Note that line 4 is an **inference** step: it uses the junction tree (or another exact/approximate method) to get the clique marginal under the current potentials.

Key Rule: MAP for MRFs

We can also do MAP to learn the unknown parameters of a UGM by adding a **prior term**:

$$\arg \max_{\theta} \left\{ \sum_i \log p(y_i | \theta) + \underbrace{\log p(\theta)}_{\text{prior term}} \right\},$$

$$p(\theta) = \mathcal{N}(\theta | \mu, \Sigma) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp \left[-\frac{1}{2} (\theta - \mu)^T \Sigma^{-1} (\theta - \mu) \right],$$

where a **Gaussian prior** with hyperparameters (μ, Σ) is often used. The prior adds a quadratic (weight-decay) term to the gradient, which is cheap; the hard part remains the unclamped term.

7 Parameter Learning: UGM (CRF)

Definition : CRF in Log-Linear Form

$$p(\mathbf{y} \mid \mathbf{x}, \mathbf{w}) = \frac{1}{Z(\mathbf{x}, \mathbf{w})} \prod_c \exp(\mathbf{w}_c^T \phi_c(\mathbf{x}, y_c)),$$

where $\phi_c(\mathbf{x}, y_c)$ is a **feature vector** derived from the **global inputs** $\mathbf{x}$ and the **local set of labels** y_c (recall Lecture 3: a CRF is an MRF whose clique potentials are conditioned on all observed variables).

Key Rule: CRF Gradient and Why It Is $O(N)$ Times Slower

The scaled log-likelihood and its gradient become

$$\begin{aligned} \ell(\mathbf{w}) &\triangleq \frac{1}{N} \sum_i \log p(\mathbf{y}_i \mid \mathbf{x}_i, \mathbf{w}) = \frac{1}{N} \sum_i \left[\sum_c \mathbf{w}_c^T \phi_c(\mathbf{y}_i, \mathbf{x}_i) - \log Z(\mathbf{w}, \mathbf{x}_i) \right], \\ \frac{\partial \ell}{\partial \mathbf{w}_c} &= \frac{1}{N} \sum_i \left[\phi_c(\mathbf{y}_i, \mathbf{x}_i) - \frac{\partial}{\partial \mathbf{w}_c} \log Z(\mathbf{w}, \mathbf{x}_i) \right] = \frac{1}{N} \sum_i \left[\phi_c(\mathbf{y}_i, \mathbf{x}_i) - \mathbb{E}[\phi_c(\mathbf{y}, \mathbf{x}_i)] \right], \end{aligned}$$

where $\mathbb{E}[\phi_c(\mathbf{y}, \mathbf{x}_i)] = \sum_{\mathbf{y}, \mathbf{x}_i} p(\mathbf{y} \mid \mathbf{x}_i, \mathbf{w}) \phi_c(\mathbf{y}, \mathbf{x}_i)$.

- The **partition function depends on the observed data** $\mathbf{x}_i$.
- This means we **cannot bring** $\mathbb{E}[\phi_c(\mathbf{y}, \mathbf{x}_i)]$ **out of the summation** over training cases, as we could for an MRF.
- We must perform **inference for every single training case inside each gradient step**, which is $O(N)$ **times slower** than the MRF case.

Definition : Gradient Approximation

We approximate the gradient using a **random subset** $\mathcal{D}' \subset \mathcal{D}$ and MCMC samples from the likelihood $p(\mathbf{y} \mid \mathbf{x}_i, \mathbf{w})$:

$$\tilde{\nabla} \mathcal{L}(w_c) = \frac{1}{|\mathcal{D}'|} \sum_{(x^n, y^n) \in \mathcal{D}'} \left[\phi_c(x^n, y^n) - \underbrace{\mathbb{E}_{y \sim p(\mathbf{y} \mid \mathbf{x}_i, \mathbf{w})} \phi_c(x^n, \mathbf{y})}_{\text{computed from MCMC samples}} \right].$$

Both approximations attack the same bottleneck: the subset $\mathcal{D}'$ reduces **how many** inference problems we solve per step, and MCMC reduces **how expensive** each one is.

Algorithm: Stochastic Gradient Descent (Nowozin & Lampert, 2013)

- 1: $w^* = \text{STOCHASTICGRADIENTDESCENT}(T, \eta)$
 - Input:** T ▷ number of iterations
 - $\eta_1, \dots, \eta_T$ ▷ sequence of learning rates (can be the same)
 - Output:** $w^* \in \mathbb{R}^D$ ▷ learned weight vector
 - Algorithm:**
- 2: $w_{\text{cur}} \leftarrow 0$ ▷ initialize parameters to 0
- 3: **for** $t = 1, \dots, T$ **do**

```

4:    $(x^n, y^n) \in \mathcal{D}'$             $\triangleright$  pick a random subset of data (often 1–3 elements),  $\mathcal{D}' \subset \mathcal{D}$ 
5:    $d \leftarrow -\tilde{\nabla}_w^{(x^n, y^n)} \mathcal{L}(w_{cur})$             $\triangleright$  gradient approximation
6:    $w_{cur} \leftarrow w_{cur} + \eta_t d$             $\triangleright$  weight update
7: end for
8:  $w^* \leftarrow w_{cur}$ 

```

Picking a small $\mathcal{D}'$ is what **reduces the amount of computation** needed to perform inference for every single training case. *Scribe's note:* here $\mathcal{L}$ is treated as a loss to be **minimized** (the negative log-likelihood), which is why the step direction is $-\tilde{\nabla}\mathcal{L}$; if $\mathcal{L}$ is the log-likelihood itself, ascend instead.

Key Rule: MAP for CRFs

We can also do a maximum a posteriori estimation of the unknown parameters of a CRF,

$$\arg \max_w \left\{ \sum_i \log p(y_i | x_i, w) + \log p(w) \right\},$$

where a **Gaussian prior** is often used for $p(w)$ (equivalently, ℓ_2 regularization of the weights).

Summary: DGM vs. MRF vs. CRF Learning

	DGM	MRF	CRF
Objective	decomposes node by node	single convex $\ell(\theta)$	single convex $\ell(w)$
Closed form?	yes (counts / least squares)	no ($Z(\theta)$ couples everything)	no
Inference in the loop	none	once per gradient step	once per training case per step
Method	MLE / MAP directly	SML (MCMC) or IPF	SGD with $\mathcal{D}'$ + MCMC
Prior	Dirichlet / normal-inverse-gamma	Gaussian on θ	Gaussian on w

Take-away: the normalizer is what decides the difficulty. A DGM is normalized locally, so learning is counting; a UGM has a **global** Z , so every gradient step needs inference – and for a CRF, Z depends on the input, so it needs one inference per data point.

8 Summary

- Computed the unknown parameters of discrete/continuous DGMs using MLE and MAP.
- Computed the unknown parameters of MRFs using stochastic maximum likelihood, and iterative proportional fitting.
- Computed the unknown parameters of CRFs using stochastic gradient descent.

Key Takeaways

1. **MLE** maximizes $\prod_i p(x_i | \theta)$; **MAP** maximizes $\prod_i p(x_i | \theta)p(\theta)$, where $p(x)$ drops out because it does not depend on θ . We maximize the **logarithm** because it is monotonic (same peak) and turns products into sums.

2. For a **univariate normal**, MLE gives the sample mean and sample variance, and maximizing the likelihood is exactly **least squares**. MAP with a **normal-inverse-gamma** prior gives $\hat{\mu} = (N\bar{x} + \gamma\delta)/(N + \gamma)$: with more data MAP $\rightarrow$ MLE, with less data MAP $\rightarrow$ the prior's maximum.
3. For a **categorical** distribution, MLE gives **normalized counts**, and MAP with a **Dirichlet** prior gives $(N_k + \alpha_k - 1)/\sum_m (N_m + \alpha_m - 1)$ – the hyperparameters act as **pseudo-counts** and remove zero-probability estimates.
4. With **complete data** a DGM's log-likelihood **decomposes into one local problem per node**, estimated from the sufficient statistics $\{x_{u,n}, x_{\pi_{u,n}}\}$: counts for tabular CPDs, and the normal equations (least squares) for linear-Gaussian CPDs.
5. For an **MRF**, the log-likelihood is convex but its gradient is $\mathbb{E}_{p_{\text{emp}}}[\phi_c] - \mathbb{E}_{p(y|\theta)}[\phi_c]$ (**clamped – unclamped**); the unclamped term needs inference, so there is **no closed form**. **SML** approximates it with MCMC samples inside stochastic gradient descent; **IPF** instead rescales each clique potential by $p_{\text{emp}}(y_c)/p(y_c | \psi^t)$ until the model's clique marginals match the data's.
6. For a **CRF**, $Z(\mathbf{x}, \mathbf{w})$ depends on each input, so inference is needed **per training case per gradient step** ($O(N)$ times slower than an MRF). **SGD** on a small random subset $\mathcal{D}'$, with MCMC for the expectation, makes this practical; MAP adds a Gaussian prior on w .

Acknowledgement: Slide content adapted from M. I. Jordan, “An Introduction to Probabilistic Graphical Models” (2002, Ch. 9); K. Murphy, “Machine Learning: A Probabilistic Perspective” (Ch. 10.4, 19.5, 19.6.3); S. Prince, “Computer Vision: Models, Learning, and Inference”; D. Koller & N. Friedman, “Probabilistic Graphical Models” (Ch. 17); D. Barber, “Bayesian Reasoning and Machine Learning” (Ch. 9.1–9.4, 9.6); S. Nowozin & C. H. Lampert, “Structured Learning and Prediction in Computer Vision” (2013).